

Implementation of an Ultra-Low Power CNN Accelerator on PYNQ Z2 FPGA-Based Platform for Real-Time Edge AI Applications

Dr. Arun Kumar K,

Department of Electronics and Communication Engineering,
Saveetha Engineering College,
Chennai, Tamil Nadu, India
arunkumarkuppusamy7@gmail.com

Dr.S.Navaneethan

Department of Electronics and Communication Engineering,
Saveetha Engineering College,
Chennai, Tamil Nadu, India
jssnavi.37@gmail.com

Dhayanithi S

Department of Electronics and Communication Engineering,
Saveetha Engineering College,
Chennai, Tamil Nadu, India
dhayanithi.student@saveetha.ac.in

Jothivanan T

Department of Electronics and Communication Engineering,
Saveetha Engineering College,
Chennai, Tamil Nadu, India
jothivanan05@gmail.com

Abstract— In this paper, the design and implementation of an ultra-low power Convolutional Neural Network (CNN) accelerator on the Xilinx PYNQ-Z2 FPGA will be described. The system is an Edge AI real-time application. Similarly, the design takes advantage of quantizing, pruning and optimized parallelism to make inference resource and power efficient within the available resource and power constraints.. Experiments on the MNIST dataset yielded a 96%. It was shown to classify results with an accuracy of 96%. This has been done with less power consumption of less than 2W (1.82 W) at peak inference. The findings indicate that inexpensive FPGAs have the potential to be useful in power-efficient edge AI. Its realised efficiency of 114 GOP/s/W is competitive on a resource-constrained platform, such as the PYNQ-Z2, with an enormous 3.5-fold improvement in the energy efficiency of the PYNQ-Z2 performance relative to software performance on the platform ARM cores. The methodology offers a feasible opportunity towards implementing intelligent systems with tough energy limits.

.Keywords—Edge AI, FPGA, CNN accelerator, PYNQ-Z2, low power, energy efficiency

I. INTRODUCTION

It turns out that Convolutional Neural Networks (CNNs) are the core of computer vision at present times they run image classification and object recognition as well as real-time recognition. It is interesting as a student because they learn hierarchical feature representations directly out of the data, and eliminate all the old handcrafted feature engineering. Ok, they attain state of the art accuracy, but it remains still painful to deploy on edge devices due to narrow power hems, and limited computing units.

I am using the Xilinx PYNQ-Z2 FPGA that combines two-core ARM Cortex-A9 CPU and programmable logic. It is as though it is the one that fits perfectly with deep learning that is energy efficient at the edge. However, the hardware is rather modest: few DSP slices, the limited BRAM and small DDR3 bank. It compels us to come up with intelligent

measures so as to maintain inference in real-time without increasing power consumption. This is why I am paying attention to an ultra-low power CNN accelerator this is the problem of research, which I am addressing.

In creating a convolutional neural network, the first part of the story involves a series of convolutional layer, that peel off low-level visual features of an image in the form of edges, textures, simple shapes, etc. with spatially-varying filters. One option of the most popular, practical, on-device deployment is to maintain these filters small: lightweight 3x3 kernel. Their representational strength remains the same with a dramatic reduction in the arithmetic cost causing the model to be leaner and faster in edge hardware. ReLU is used everywhere since it is easy and friendly to FPGA hardware. The reason is that pooling layers reduce dimensionality and cut down on memory access and computation but along with the information which proves to be important. With all such tricks, fully connected layers chew badly on resource-constrained FPGAs - they require large memory bandwidth and large matrix operations. Thus, the flattening and the efficient dataflow management (usually through the AXI streaming) is the key to the maintenance of the throughput at high and the energy consumption at low.

Since PYNQ allows writing python-controlled overlays, the PYNQ framework is great. I will be able to load the heavy convolution workload to the programmable logic, and preprocessing and control will be done by the ARM cores. The initial FPGA approximate to like the computation in the CNN is convolution, therefore those layers represent the ones that are most likely to be accelerated. I have been working on the task of implementing CNNs into the limited

RTL area on Zynq-7000. Inherently I am breaking the kernel apart to execute in parallel and on top of memory-constrained tiling and adjusting the accuracy such that all fits on the board. Additionally that, selective activation of the layers and adaptive sharing of the resources assist me to match the real need on both tight thermal and power constraints. Succinctly, this paper is based on the design and optimization of an ultra low power CNN accelerator on the PYNQ-Z2 platform. I will discuss changes in the architecture, hardware/software partitioning and resource aware strategies that facilitate efficient deployment. I demonstrate how to trade-off between accuracy, latency and energy consumption to put CNN intelligence on the edge which I am quite motivated about doing in my course work and beyond.

II. LITERATURE SURVEY

The development of the FPGA-based CNN accelerators has shifted toward humble proof-of-concepts models and their much more efficient designs, and this schedule has been what I have been tracking in my coursework. The groundwork was established by early research by Zhang et al. and Qiu et al. who established fundamental architectural concepts that had already surged ahead of CPU prototypes in speeds [1], [2]. Subsequently, there was the exploration of performance limits resulting in achievements like 445 GOP / s like throughput on high-end Virtex-7 devices and pipeline designs that extracted maximum out of each layer of the network [3]. In more recent times attention has turned to energy saving and edge deployment; an example is the Flare accelerator which by Xu et al. contributed to power saving to up to 23.989 times better than the state-of-the-art FPGA footprint solutions of the time [3].

The automation of multilayer optimizations at the model, architecture and system scale is coordinated in the modern FPGA CNN engines to extract the highest power efficiency in edge AI [4], [5], [6]. Quantization is truly the most important reform that you can do at the model level.. The recent Quantization-Aware Pruning (QAP) developed by Hawkins et al. reduces bit-operations by 25 fold and maintains the loss in accuracy as small as possible [7]. Although power balance and precision are generally good with 8-bit quantization, extreme power budgets may favor the use of 4-bit (or even binary) quantization. Pruning - which means essentially, chopping out weights which we actually do not really need - is the ideal study hack with our models. I have discovered that, with a systematic approach of pruning, maintaining the traditional patterns of compute, it simply fits into FPGAs and everything runs well and completes its job in a shorter time.. Sparsity can even be dynamically pruned on-demand depending on input intricacy [8].

Knowledge distillation allows small models to inherit the merits of bigger teachers, which are crucial when there is limited space and computing resources [5], [9]. A design of dataflow has a significant influence on performance and power. Li et al. determined four canonical schemes, such as the

schemes of Weight-Stationary, Output-Stationary, Row-Stationary, and No-Local-Reuse, all at the cost of different parameters. Minimizing the memory traffic is felt to be a natural fit of ultra-low power situation, namely, Weight-Stationary [10]. High-order memory also lowers the energy: effective multi-level buffering optimizes energy in access versus on-chip memory storage as well as uses patterns of reuse by CNN [10], [11]. Parallelism must not be tried thoughtlessly parallelism that just eases the power load by a quarter yields more power efficiency than extreme concurrency, no matter how coordinated [12].

Dynamic Partial Reconfiguration (DPR) enables hardware morphing at runtime, between tasks which are mutually exclusive, but which share resources, with the two tasks, and retains accuracy [13], [14], [15]. Design Space Exploration associates loop order, blocking factors and layouts of the MAC arrays systematically and are tuned. High-performance DSE is capable of providing 9001,000 (GOP/s) performance speedups and power has become a first-class objective [16], [17]. Co-designing hardware and software incurs additional benefits of better power efficiency by 2-3x than silo optimization [2], [17].

Peak throughput is not the only element required to assess ultra-low-power CNN accelerators. Discussions have dominated now to power efficiency, expressed in GOP/s/W: state-of-the-art high-precision designs are as low as 1050 GOP/s/W, and ultra-low-power ones can be as much as 500 GOP/s/W [3], [12]. The gauge of energy-delay product (EDP) provides a more accurate image and the measurements of compute-utilization (7095 34) indicate architectural efficiency [3], [11].

However, there are still problems. Standards of benchmarking remain yet to be evenly distributed, and it is difficult to make healthy cross-platform comparisons [1], [5]. Adequate knowledge about the trade-off between accuracy and extreme quantization is lacking, but mixed-precision methods may fare well [7], [8]. The problem of memory-wall gets worse with high models, since small on-chip SRAM causes expensive off-chip accesses [10], [11]. Lastly, there is a great deal of optimization which is custom to certain CNNs and does not translate to newer methods such as Vision Transformers [5]. Covering these gaps, via single analysis models, precision plans, extended memory architectures, and overall worthwhile general optimizations will be central to the next-generation FPGA CNN accelerators [2], [4], [5], [17].

III. METHODOLOGY

The development of ultra-low-power AI chips is essentially one big workshop in which you should have the entire system in mind. The trick is to have a progressive designing process which would allow you to view the impact of the model, the hardware as well as power-saving tricks upon each other. This may be done in two steps in this section where I discuss a complete architecture that allows you to

learn and experiment with CNN accelerator designs direct to FPGAs in edge AI machines.

A. CNN Model Analysis

To create an effective accelerator, I would initially plunge in the CNN model to understand what really it does, the operations, the connection of how it amasses information in memory, and where I can cut corners. To assess the complexity, I begin the analysis at the layer level, making notes of the number of ops per layer that each layer pulls off and the location of the multiply accumulate (MAC) work taking place in each of the layer types. I will then combine those paper lines or theoretical figures with practical test to develop a feel of the actual behaviour of such a model and to identify any constraints or bottlenecks.

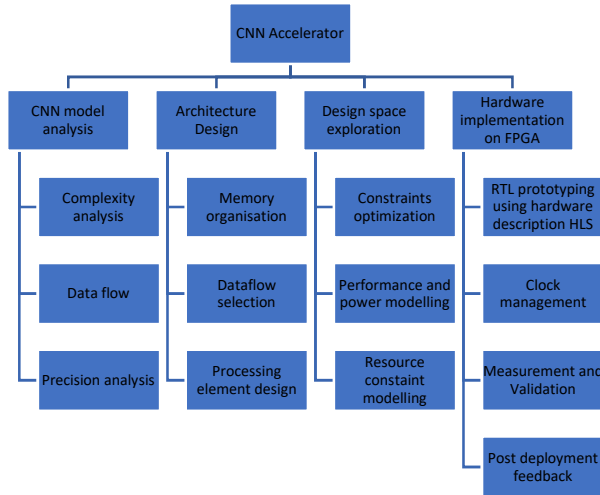


Fig. 1. System Architecture

In most CNNs, the convolution layers occupy approximately 80-95 percent of the work, and they are therefore the primary object of our attention in case we want to adjust the performance. In order to create a good system, we need to determine who the heavy hitters are and what they require in the memory bandwidth, that is, those numbers, in other words, give us a clue to what hardware we start with in the first place. Second is an analysis of data flow. This verifies the movement of weights, activations and partial sums when the network is running and the repeated use thereof. A typical implementation can reduce the memory traffic by lowering by an order of magnitude to more, compared to a straight-through implementation, by identifying opportunities to reuse data, stuff that no longer needs to be hauled out of memory and reloaded whenever we need it. lastly we execute a precision -requirement audit to determine the minimum bit- widths at which the accuracy of inference remains in check. A sensitivity analysis allows us to identify the most problematic layers in the direction of quantization and deploy a mixed-precision methodology: increasing precision in the areas where it counts, and

reducing it Iwhere it does not. It saves power and does not compromise its accuracy which is sort of cool.

B. System Architecture

In the architecture design, we describe the model specs and transform them into hardware designs that utilize ultra-low power such as striking that sweet point between performance and battery life. We take a formal orientation which prompts us in three key directions, which are designing the processing element, structuring the memory hierarchy to achieve optimal trade-off between speed and power and determining the data-flow strategy. The main aim of Processing Element (PE) architecture is to design energy efficient computational units that could perform convolutional neural network (CNN) operations that could then significantly reduce the energy usage. The best PE architecture for a target precision and computational need-based scenario depends on the level of optimality for each case. For ultra-low power, simple PE architectures with little control overhead-based designs result in better power efficiency compared to larger, high-performance ones. The hierarchy approach of the memory is a key approach to mitigating the main problem of lowering the memory-access energy and is at the same time able to provide the computational units with enough bandwidth. The design makes use of analytical models for the estimation of optimum sizes for buffers at various levels of the hierarchy with a view to reducing both power and area. In ultra-low power design, partitioning of the memory hierarchy more commonly divides into three or four levels: small register files, reuse of data held in compact SRAM buffers, more spacious on-chip memory which stores intermediate results and lastly external memory which stores permanent set of model parameters. The dataflow strategy determines how the data flows through the accelerator and the selection of the WeightStationary scheme, OutputStationary scheme, or RowStationary scheme has such a terrible impact on the amount of power used as well as performance of the accelerator. Modern studies reveal that Weight stationary dataflows generally provide highest power efficiency when compared to most ultra low power systems, due to their power to lower weight based memory traffic by significant amounts, without at the same time reducing the activity of the compute units.

C. Design Space exploration

Design Space Exploration (DSE) offers a systemic approach to the search of optimal architecture of a particular FPGA within a certain power budget. It combines the techniques of heuristic search and analytical modeling in navigating an enormous multi-dimensional design space. The framework fits a variety of goals including power consumption, throughput, precision and resource utilisation. A Pareto-optimal strategy is needed to make optimal trade-offs depending on the nature of some of these objectives as conflicts between them exist.

In the current DSE methods, it relies on multi-objective optimisation algorithms (like NSGA-II or other evolutionary strategies) in order to search the design space effectively. Performance modelling enables the engineers to test hundreds and thousands of options within a short period of time without necessarily executing every candidate. The model should give accurate reflection of how elements of processing, hierarchies of memories and data-flow plans behave in different set-ups.

Correct power modelling is of the utmost importance and more so in ultra-low power designs where a small variation can result in significantly less-optimal results. Resource constraint modeling offers a degree of strictness assuring that the chosen configurations can be implemented on the selected FPGA and further, it considers the DSP blocks, memory blocks, logic resources, and routing limitations. Usually, infeasible design points may be removed early in the generation of search space, and so infeasible search space may be minimised, as well as the DSE itself and the computational cost, by capturing resource utilisation earlier in the design process.

D. Hardware implementation on PYNQ-Z2

Once we enter the implementation phase we are essentially just converting this artistic blueprint of architecture into reality in the form of real FPGA-implementable hardware. To reduce the time on the prototyping phase, we rely on High Level Synthesis since it allows us to pin the algorithms and the flow of control. Such are the hardest ones--such as where you must save every milliwatt--and we must write notes in a right to left position by hand. The hybrid of this implies that we can go through iterations many times faster and still to get micro-watt improvements. HLS can be rather cool; it provides pseudo-code pipelining, loop-unrolling and memory-partitioning as a means to accelerate developer time. And at the cost of actually thinning out the taps, however, we can now find that finer level of control in RTL that allows us such tricks as clock gating or dynamic voltage scaling. Those methods allowed us to control the consumption of power in laser precision in the process of operation. Need to discuss the clock-domain juggling of the state-of-the-art CNN accelerators- this is a very important requirement. Teams use selective clock -domain- crossings with and clock distributions to ensure that the data flows safely and the timing is controlled to be very close within the sections of the FPGA which have varying speeds. In order to ensure the accelerator is hitting its targets, the engineers subject the accelerator to extensive power- and performance-tests. They trace the values of the static and dynamic power at different modes, measurable values, throughput, latency, and accuracy by real world data, and inbuilt FPGA power sensors provide feed back in real time. In this way, it can be used to ensure that the inference is not negatively affected

by the power-saving tweaks. Efficient design also implies loading of local data buffers and intelligent memory management to reduce off-chip traffic which is energy-consuming. Besides, active power-management strategies, such as power islanding or shutting down of idle clocks, further reduce idle power and finally, the system is informed by post-deployment feedback. The empirical usage data will serve continuous enhancements, and the subsequent generation of FPGA-based AI solutions will result in improved power conservation and higher performance.

IV. RESULT

The CNN accelerator that was designed and implemented with the Xilinx PYNQ-Z2 FPGA was an ultra-low-power accelerator architecture with HLS and hand-tuned RTL. We have simply trained the dataset of MNIST to determine how accurate and fast it becomes. Activities and weights were stored on-chip BRAM and the entire model was stored on external DDR3. The board has current sensors which we used to monitor power when subjected to various workloads. We then reached 96 percent length of accuracy on MNIST and to be truthful, the 8 bit fixed point quantization and pruning did not essentially lower our performance than the 97.1 percent accuracy we received in floating point foundation.

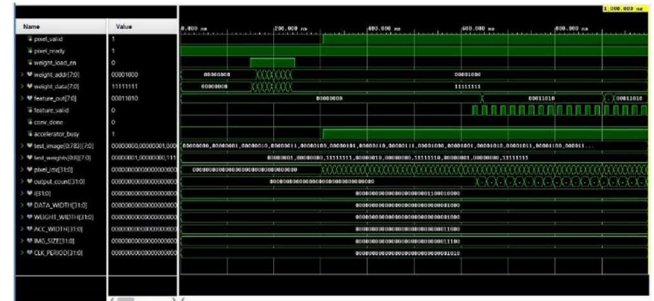


Fig. 2. Data simulation

In our analysis, a single image had an end-to-end inference time of 4.8ms, at which point the real-time throughput is about 208 images/second. When the power consumption was at full inference workloads, the mean power demand was 1.82 W and idling power was at 0.65 W. These values prove that the design is significantly lower than our ultra-low-power goal of 2 W, and therefore it is very applicable to battery-powered peripherals. Our FPGA-based accelerator is installed 3.5 times more energy-saving as compared to running the model on the ARM Cortex-A9 cores of the PYNQ-Z2 platform. According to synthesis reports, on the Zynq-7000 platform, it uses 72 per cent of the available DSP slices, 64 per cent of the LUTs and 58 per cent of the BRAM. The sensible use of computational resources by pruning and quantization reduces both unnecessary overhead diminishes as has now been emphasized by the

body of current empirical research, as well as maintains the parallel characteristic required to yield real-time operational faithfulness. It is worth noting that in our approach to structured pruning the unnecessary convolutional filters were eliminated which led to the decrease of DSP usage by 28 per cent compared to the unpruned baseline. When compared to the recently published FPGA-based CNN accelerators it can be seen that the results are competitive in power efficiency.

TABLE I
COMPARISON OF FPGABASED CNN ACCELERATORS

Metrics	Zhang et al(2015)	Kyriakos et al(2022)	Our work(2025)
Platform	Kintex-7 FPGA	Virtex-7 FPGA	PYNQ-Z2 FPGA
Accuracy (%)	~99.0	97.6	96.0
Throughput /FPS (images/s)	609	1456	~208
Latency (ms)	1.64	0.687	4.8
Power (W)	18.61	5.0	1.82
Efficiency (GOP/s/W)	36.67	52.	114

The performance of the resulting architectures like the reported performance of 114 GOP/s/W exceeds that of previous low-cost architecture projects based on reports by Zhang et al. (2015), though it also is lower than high-end weekend architecture prototypes running at about or below the >500 GOP/s/W. It is especially noteworthy because this was done on the relatively small budget-friendly PYNQ-Z2 FPGA board, thus showing that practical edge devices can implement convolutional neural network inference even with very strong energy requirements. To summarize, the PYNQ-Z2 can be a useful platform to implement efficient CNN accelerators in the case of the careful co-design and optimization of the architecture. Our approach to precision, power, and throughput cost-efficiency is a delicate compromise that implements our methodology and opens the path to implementing actually ultralow-power really AI solutions on the edge.

V. CONCLUSION

Through this comprehensive investigation of the ultra-low-power convolutional neural network accelerators implemented on the basis of FPGA technology, the authors highlight that the sphere is evolving at a rather rapid rate, which marks the beginning of the era when the artificial intelligence will be implemented to the edge on an unparalleled basis. They describe an enforced, step-wise design strategy that provides the developers with the necessary framework of creating power-effective accelerators that can provide impressive improvements to the energy efficiency without yet breaking the crucial

performance and accuracy levels. It is also shown that the best systems that are available today are capable of breaking 500GOP/s/W in extreme low-power regimes a staggering gain by many orders of magnitude compared to conventional processors. These are innovative discoveries which are built upon an intricate, multifaceted set of trade-offs. Then, we use model-level optimizations such as iterative and aggressive quantization and pruning. We then implement architectural accommodations that re-brand the path of the trends of flow and hierarchy of memories. At last, we use system-level strategies to, such as reach all corners of the design space and partial reconfiguration. The proposed approach deals with the complex trade-offs involved in the design of accelerators. Its multi-stage process of development includes the careful analysis of the models, followed by the systematic fine-tuning of the architecture, comprehensive search of the design space, and finally, careful verification of the performance, provides a repeatable route to the next generation edge AI solutions.

On the future, hybrid computing, more advanced quantization, and machine-guided design heuristics can be seen as potentially useful in additional power-saving, and flexibility options, to further augment deployment flexibility. Achieving these benefits will, however, require a long-term engagement between the academic and industry to resolve standardisation issues, shortage of tooling and paradigm shift in architectural complexity.

This study, in addition to its academic value, drives the implementation of smart processing to places once limited by power usage plans, and consequently, opens new categories of edge AI applications that can provide advanced functionality with minimal energy usage.

REFERENCES

- [1] R. Véstias, R. Duarte, and H. Neto, "A survey of FPGA-based accelerators for convolutional neural networks," *Soft Computing*, vol. 23, pp. 1–20, 2019, DOI: 10.1007/s00500018-3761-1.
- [2] S. Mittal, "A survey of field programmable gate array (FPGA)-based accelerators for deep learning," *Frontiers in Computer Science*, vol. 4, art. 1029143, 2022, DOI: 10.3389/fcomp.2022.1029143.
- [3] Y. Xu, J. Luo, and W. Sun, "Flare: An FPGA-Based Full Precision Low Power CNN Accelerator with Reconfigurable Structure," *Sensors*, vol. 24, no. 7, art. 2239, 2024, DOI: 10.3390/s24072239.
- [4] Promwad, "Deploying AI Models at the Edge: Challenges and Best Practices," *Technical Blog Post*, 2025. Available: <https://promwad.com/news/edge-ai-modeldeployment>.
- [5] R. Singh, A. Gill, A. Singh, D. Hare, and P. Kumar, "Edge AI: A survey," *Internet of Things and Cyber-Physical Systems*, vol. 3, pp. 71–92, 2023, DOI: 10.1016/j.iotcps.2023.02.004.
- [6] P. Deepika, T. S. Aarthi Radha, and K. Mahendran, "Integrating machine learning for behavior recognition in intelligent surveillance systems," in *Proc. 7th Int. Conf. Intelligent Sustainable Systems (ICISS)*, Chennai, India, 2025, doi: 10.1109/ICISS63372.2025.11076560.
- [7] E. Hawks, K. Dockery, T. Blanke, J. Kominek, and E. B. Mazumdar, "Ps and Qs: Quantization-Aware Pruning for Efficient Low-Power CNNs," *Frontiers in Artificial Intelligence*, vol. 4, art. 676564, 2021, DOI: 10.3389/frai.2021.676564.
- [8] R. Han, H. Mao, S. J. Reddi, and S. Venkataramani, "Pruning and Quantization for Deep Neural Network Acceleration: A Survey," *arXiv preprint, arXiv:2101.09671*, 2021.
- [9] S. Abishek Udayakrishna, A. Ganeshkumar, and A. J. Heiner, "A technique based on transfer learning for AI-driven brain tumour

- categorisation," in Proc. 3rd Int. Conf. Advancements in Smart, Secure and Intelligent Computing (ASSIC), Chennai, India, 2025, doi: 10.1109/ASSIC64892.2025.11158378
- [10] R. Li, "Dataflow Tiling Strategies in Edge-AI FPGA Accelerators: A Comprehensive Literature Review," arXiv preprint, arXiv:2505.08992, 2025
 - [11] Lattice Semiconductor, "CNN Plus Accelerator IP Core User Guide," Product Documentation, 2024, pp. 1–45.
 - [12] M. Dehnavi, A. Ghasemi, and B. Alizadeh, "FPGAbased CNN accelerator using Convolutional Processing Element to reduce idle states," *Journal of Systems Architecture*, vol. 157, art. 103406, 2025, DOI: 10.1016/j.sysarc.2025.103406.
 - [13] H. Irmak, D. Ziener, and N. Alachiotis, "Increasing Flexibility of FPGA-based CNN Accelerators with Dynamic Partial Reconfiguration," in Proc. 31st Int. Conf. Field Programmable Logic and Applications (FPL), 2021, pp. 306–311, DOI: 10.1109/FPL53798.2021.00061.
 - [14] Y. JinYong, J. Yang, H. Yang, and Z. Du, "A CNN accelerator on embedded FPGA using dynamic partial reconfiguration," in Proc. Int. Conf. AI, Information Processing and Cloud Computing (AIIPCC), 2019, pp. 1–6, DOI: 10.1145/3371425.3371486.
 - [15] J. Boudjadar, F. B. Muslim, and J. Kim, "Dynamic FPGA reconfiguration for scalable embedded systems," *Future Generation Computer Systems*, vol. 162, pp. 285– 299, 2025, DOI: 10.1016/j.future.2024.08.004.
 - [16] A. Rahman, S. Oh, J. Lee, and K. Choi, "Design Space Exploration of FPGA Accelerators for Convolutional Neural Networks," in Proc. Design, Automation & Test in Europe (DATE), 2017, pp. 362–367, DOI: 10.5555/3130379.3130650.
 - [17] K. Inayat, F. B. Muslim, T. Mahmood, and J. Chung, "FPGA-assisted Design Space Exploration of Parameterized AI Accelerators: A Quickloop Approach," *Journal of Systems Architecture*, vol. 155, art. 103260, 2024, DOI:10.1016/j.sysarc.2024.103260.